Table 1: Causes and impacts of IoT system failures

Factor	Description	Impact
Device limits	Low memory and processing power	Fail under unexpected loads
Network issues	Unstable WiFi, Bluetooth, LoRaWAN	Lost packets, delays
Hardware and software diversity	Many chips, OS, protocols	Hard to maintain stability
Firmware	Bugs, update failures	Freezes, lockouts, downtime
Operational factors	Battery, sensors, extreme conditions	Malfunctions, inaccurate readings
Consequences	Industrial, healthcare, smart home	Financial losses, safety risks
Mitigation	Chaos engineering	Predict failures, improve resilience

What is chaos engineering?

System testing using chaos engineering involves intentionally inducing system failures to see how they react. Chaos engineering began in the early 2010s when Netflix engineers experienced continuous outages on Amazon Web Services. The platform was new and still evolving. To address this, the Netflix team created Chaos Monkey, a tool that could simulate unexpected system failures in production environments. Chaos Monkey employed a straightforward but effective technique. It randomly shut down servers to see how the system behaved. This taught engineers to never take system reliability for granted. Through controlled failure tests, the team was able to identify weaknesses in the system, and this helped them prevent future real-world breakdowns.

The development of Chaos Monkey gradually expanded into a complete and structured approach known as chaos engineering. Large tech firms like Microsoft, Google and Amazon have adopted this strategy. Smaller businesses like Shopify and Atlassian, too, have begun conducting controlled failure tests. Shopify uses its online store to create artificial database failures, which help the platform sustain its operations during times of heavy user traffic. These controlled failure tests enable businesses to improve their recovery processes.

Chaos engineering is mostly used in cloud environments because it works very well there. However, it is more difficult to apply to IoT and edge computing systems. IoT devices are physical, often located in remote places and sometimes perform critical tasks. This makes managing them even more challenging. Restarting cloud servers using scripts is usually simple. But rebooting medical devices like pacemakers, industrial robots or warehouse sensors is much more complex and can be dangerous. Resetting edge devices also takes longer because system failures often have immediate physical outcomes.

Chaos engineering in IoT systems has both benefits and challenges. Engineers need to design methods to test failures safely without harming devices. The testing process aims to detect equipment breakdowns while developing systems that function during actual operational conditions. The proven cloud software methods of chaos engineering enable organisations to meet the requirements of edge devices.

A commercial home appliance company can use chaos engineering principles to test their IoT cloud platform for weaknesses. This testing will help them improve system reliability and make the platform more robust. The open source µChaos software (used to simulate faults in smart sensors) for the ZephyrOS real-time operating system (commonly used in small IoT devices like wearables, smart home controllers, etc) lets engineers perform fault injection tests. These tests replicate failures and help engineers build tough devices.

The proactive approach of chaos engineering helps organisations to detect system vulnerabilities. This enables them to verify their systems' ability to operate during failures and recover functionality after breakdowns. Chaos engineering for IoT is difficult but essential for building reliable edge devices.

Bringing chaos engineering to IoT and edge devices

As we saw earlier, the practice of chaos engineering in IoT systems requires engineers to create intentional failures at the edge. The testing process checks whether systems can stay stable when unexpected failures happen. Engineers perform tests by making sensors give unrealistic readings, like -200 degrees Celsius, to see if smart HVAC systems (heating, ventilation and air conditioning systems used for climate control in buildings) handle these situations properly. They also reduce network traffic to test whether industrial equipment can recover automatically. They sometimes restart wearable health trackers during important data transfers. They watch to see if the devices recover while keeping important information safe. Engineers also run power drain tests to quickly deplete batteries. This shows whether users get shutdown warnings before the device stops suddenly.

Researchers at the University of California, Berkeley, employed Raspberry Pi clusters to create sensor networks that replicated the operational